



PROJECT REPORT COMPUTER PROGRAMMING

GROUP MEMBERS:

Name	Roll NO
Ahmad Hassan (Group Lead)	2501588
Obaid Ur Rehman Saad	2501576
Hafiz Muhammad Saad Habib	2501525

SESSION:
Spring 2026

**DATE OF REPORT
SUBMITTED: 28-May-2026**

GRADE/POINTS:

Department of Mechatronics and Biomedical Engineering

Library Management System Using C++ and CSV File Handling with ESP32 Authentication

Objectives:

- To develop a Library Management System using C++ programming.
- To manage library records digitally using CSV file handling.
- To reduce manual paperwork and improve record accuracy.
- To implement secure access using ESP32 authentication.
- To understand practical applications of programming and file management.

Abstract:

The Library Management System is a computer-based application developed using C++ programming language and CSV file handling techniques to automate the management of library records. The system is designed to reduce the difficulties associated with manual record keeping such as data redundancy, data loss, slow searching, and inefficient book management. The project provides an easy and efficient method for storing, updating, searching, issuing, returning, and deleting book records digitally.

The application uses CSV files as a lightweight database for permanent storage of information related to books including Book ID, title, author name, genre, issued status, and available copies. The system also integrates ESP32-based authentication to provide hardware-level security. Access to the software is granted only when the ESP32 device is properly connected and verified through serial communication.

The project demonstrates important programming concepts such as file handling, functions, loops, conditional statements, string manipulation, data validation, and serial communication using the Windows API. The menu-driven interface allows users to interact with the system easily and perform operations efficiently.

This project successfully combines software and hardware technologies to create a secure and user-friendly library management solution. It improves record management speed, reduces paperwork, ensures data integrity, and provides practical experience in real-world system development using C++ and embedded systems.

Introduction:

The Library Management System is a computer-based application developed in C++ to manage the records of books in a library. The system allows the librarian to perform different operations such as adding books, searching books, issuing books, returning books, and deleting records.

The project uses CSV file handling to permanently store the data of books. It also includes ESP32-based hardware authentication to improve system security. Before accessing the system, the ESP32 device must be connected successfully; otherwise, access is denied.

This project demonstrates important programming concepts including:

- **File handling**
- **Functions**
- **Conditional statements**
- **Loops**
- **String manipulation**
- **Data management**
- **Serial communication with ESP32**

The system provides a simple text-based menu interface that makes the library operations easy and efficient.

Problem Statement:

Managing library records manually is difficult, time-consuming, and prone to errors such as data loss, duplication, and inefficient record handling. Small libraries often lack secure and organized digital systems for storing, searching, issuing, and returning books. This project solves these problems by developing a computerized Library Management System using C++ and CSV file handling with ESP32-based authentication to provide efficient, secure, and reliable management of library records.

Technologies Used:

Technology	Purpose
C++	Main programming language used for system development
CSV Files	Used as database storage for records
ESP32	Provides hardware-based authentication
File Handling	Stores and retrieves data from files
Windows API	Enables serial communication with ESP32

C++:

C++ was used as the primary programming language for developing the Library Management System. It provides fast execution, structured programming, file handling support, and efficient memory management. Different programming concepts such as functions, loops, conditional statements, and string manipulation were implemented using C++.

CSV Files:

CSV (Comma Separated Values) files were used as a lightweight database for storing library records permanently. Each record contains information related to books such as Book ID, title, author, genre, and availability status. CSV files are simple, easy to manage, and suitable for small-scale applications.

ESP32:

ESP32 is a microcontroller used for authentication and security purposes in the project. Before accessing the system, the program checks communication with the ESP32 device through the COM port. Access is only granted if the correct authentication signal is received.

File Handling:

File handling techniques were implemented to read, write, update, and delete records from the CSV database. This allows permanent storage of library information even after the application is closed.

Windows API:

The Windows API was used to establish serial communication between the C++ application and the ESP32 device. It helps in detecting COM ports and receiving authentication signals from the hardware device.

Features of the System:

The Library Management System provides multiple features to simplify library record management and improve efficiency. The system uses a menu-driven interface that allows users to perform different operations easily.

Main Features

- Add new books into the database
- Search books using Book ID
- Issue books to students
- Return issued books
- Delete book records
- Automatic CSV database creation
- Duplicate Book ID checking
- Hardware-based ESP32 authentication
- Permanent data storage
- User-friendly text-based interface

The system reduces manual paperwork and provides fast access to library records while maintaining data accuracy and security.

Data-base Structure:

The project stores all records inside a CSV file named:

Library02.CSV:

The CSV file acts as the main database of the system. Each row represents one book record while each column stores a specific type of information related to the book.

Database Fields:

Field Name	Description
Book ID	Unique identification number of the book
Title	Name of the book
Author	Author of the book
Genre	Category or subject of the book
Total Copies	Total copies available in library
Available Copies	Remaining available copies
Issued To	Student ID of issued user
Issue Date	Date on which book was issued
Due Date	Return deadline of the book

The database structure allows efficient storage and retrieval of records while maintaining proper organization of information.

Methodology:

The development of the Library Management System was completed in multiple stages. First, the requirements of the project were analysed to identify the important functions needed in the system such as adding books, searching records, issuing books, returning books, and deleting records.

After requirement analysis, the CSV database structure was designed to store book information properly. Different fields were selected to maintain complete records of books and issued students.

In the next stage, the complete program was developed using C++ language. Separate functions were created for each operation to improve code organization and simplify debugging. File handling techniques were implemented to store records permanently inside the CSV file.

After completing the software portion, ESP32 authentication was integrated using serial communication through the Windows API. The system checks the hardware connection before granting access to the user.

Finally, the complete project was tested multiple times to remove errors and verify correct working of all operations.

Working Principle:

The Library Management System works through a menu-driven interface where users can select different library operations according to their requirements. When the program starts, the system first checks the ESP32 authentication device through the COM port.

If the ESP32 device is connected and the correct authentication signal is received, access to the system is granted. Otherwise, the application closes automatically to prevent unauthorized access.

After successful authentication, the main menu is displayed. The user can perform operations such as adding books, searching records, issuing books, returning books, and deleting records.

All data is stored inside the CSV database file. Whenever an operation is performed, the file is updated automatically. The system also checks for duplicate Book IDs and verifies available copies before issuing books to ensure proper data management and accuracy.

Code Explanation:

The Library Management System was developed using C++ programming language with file handling and ESP32 serial communication. The code is divided into multiple functions where each function performs a specific task related to library management operations.

Header Files

Explanation of Header Files

- `iostream` → Used for input and output operations.
- `fstream` → Used for file handling operations.
- `windows.h` → Used for serial communication with ESP32.
- `sstream` → Used for string conversion and processing.
- `string` → Used for string manipulation operations.
- `cstdlib` → Used for standard functions like `atoi()`.

Global Variables

Const string DB_FILE

- library02.csv

Const string HEADER

- Book ID
- Title
- Author
- Genre
- Total Copies
- Available Copies
- Issued To
- Issue Date
- Due Date

Explanation:

These global variables define the database file name and the header row used in the CSV database file. The variables help maintain consistency throughout the program.

Function: get Field ():

Purpose:

This function extracts a specific field from a CSV record.

Working:

- Reads the CSV line character by character
- Detects commas as field separators
- Returns the required field value

This function helps in accessing individual data fields from database records efficiently.

Function: initialize-Database ():

Purpose:

Checks whether the CSV database file exists or not

Working:

- If the file already exists, the function does nothing
- If the file does not exist, a new file is created
- The header row is automatically written into the file

Importance:

This function prevents database errors during program execution and ensures proper initialization of the system.

Function: is-Unique ():**Purpose:**

Checks whether a Book ID already exists in the database.

Working:

- Reads all records from the CSV file
- Compares existing Book IDs with the entered ID
- Returns true if the ID is unique
- Returns false if the ID already exists

Importance:

This function prevents duplicate book records inside the database.

Function: append-Record ():**Purpose**

Adds a new book record into the CSV database.

Working:

1. Extracts the Book ID
2. Checks ID uniqueness
3. Opens the file in append mode
4. Writes the new record into the file

This function permanently stores book information in the database.

Function: search-By-ID ():

Purpose:

Searches for a book using its Book ID.

Working:

- Reads each record from the CSV file
- Compares Book IDs
- Displays complete information if the record is found

Output Displayed:

- Book ID
- Title
- Author
- Genre
- Available Copies
- Due Date

This function allows fast searching of book records.

Function: update-Record ():

Purpose:

Updates or deletes records inside the database.

Working:

1. Creates a temporary file
2. Copies all existing records
3. Replaces the matched record with updated data
4. Renames the temporary file as the original database

Importance:

This method maintains data integrity and prevents corruption of the database file.

Function: issue-Book ():

Purpose:

Issues a book to a student.

Working:

1. Takes Book ID as input
2. Checks available copies
3. Decreases available copies count
4. Stores student issue information
5. Updates the database file

Condition:

A book can only be issued if copies are available in the library.

Function: return-Book ():

Purpose:

Returns a previously issued book.

Working:

1. Finds the issued book record
2. Increases available copies count
3. Clears issue-related information
4. Updates the database file

This function updates the system after successful book return.

Function: add-Book ():

Purpose:

Adds a completely new book into the library system.

Working:

- Takes book details as user input
- Creates a CSV record
- Stores the record permanently inside the database file

This function is used for maintaining new library entries.

Function: delete-Book ():

Purpose:

Deletes a book record from the database.

Working:

The function calls update-Record () with empty data to remove the selected record from the CSV database.

Function: authenticate-ESP32 ():

Purpose:

Provides hardware-based authentication and security.

Working:

1. Connects to ESP32 through COM3 port
2. Sends an authentication request signal
3. Reads the authentication key from ESP32
4. Grants access if the correct key is received

Security Feature:

If the ESP32 device is not connected or authentication fails, the system closes automatically to prevent unauthorized access.

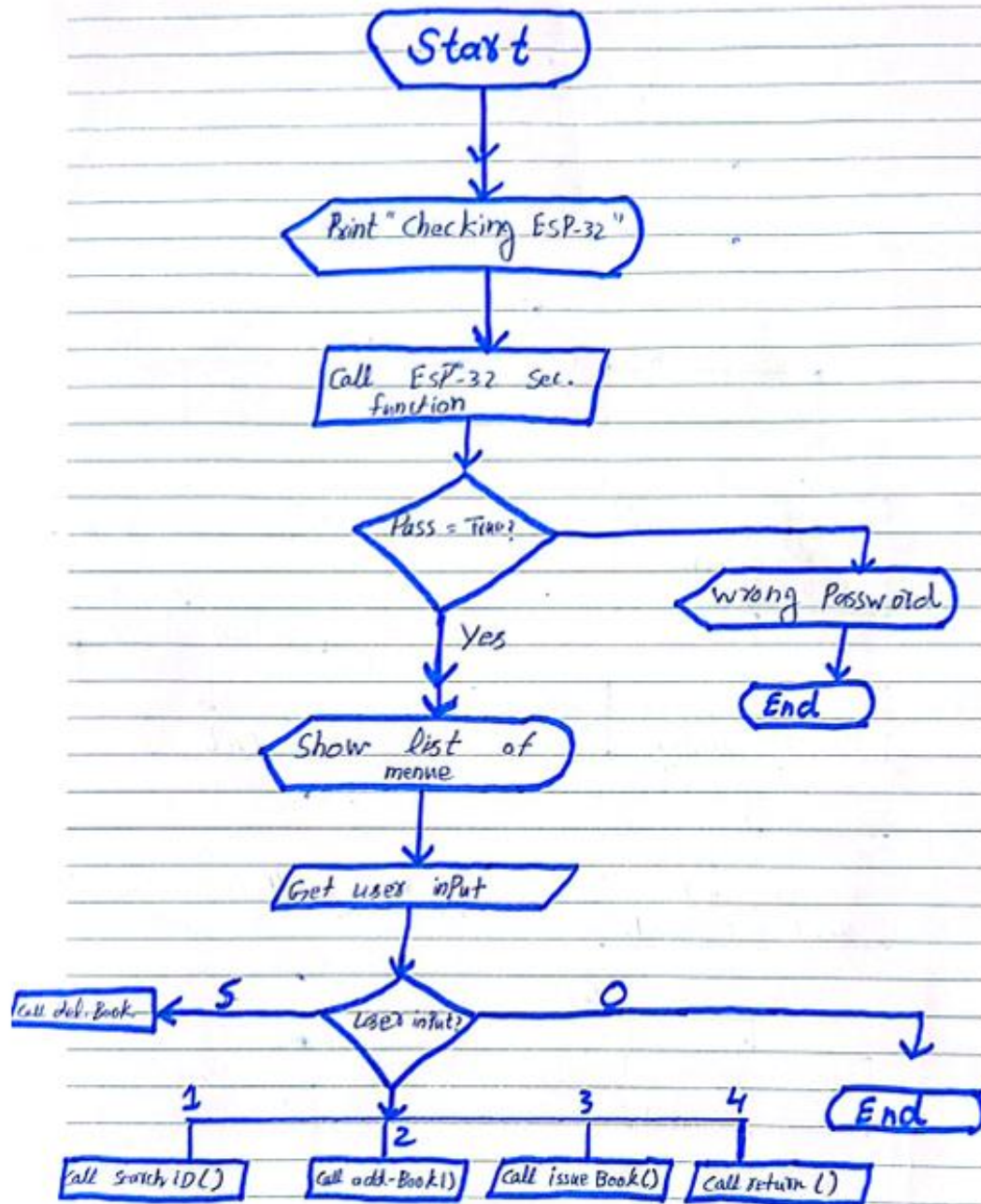
Block Diagram:



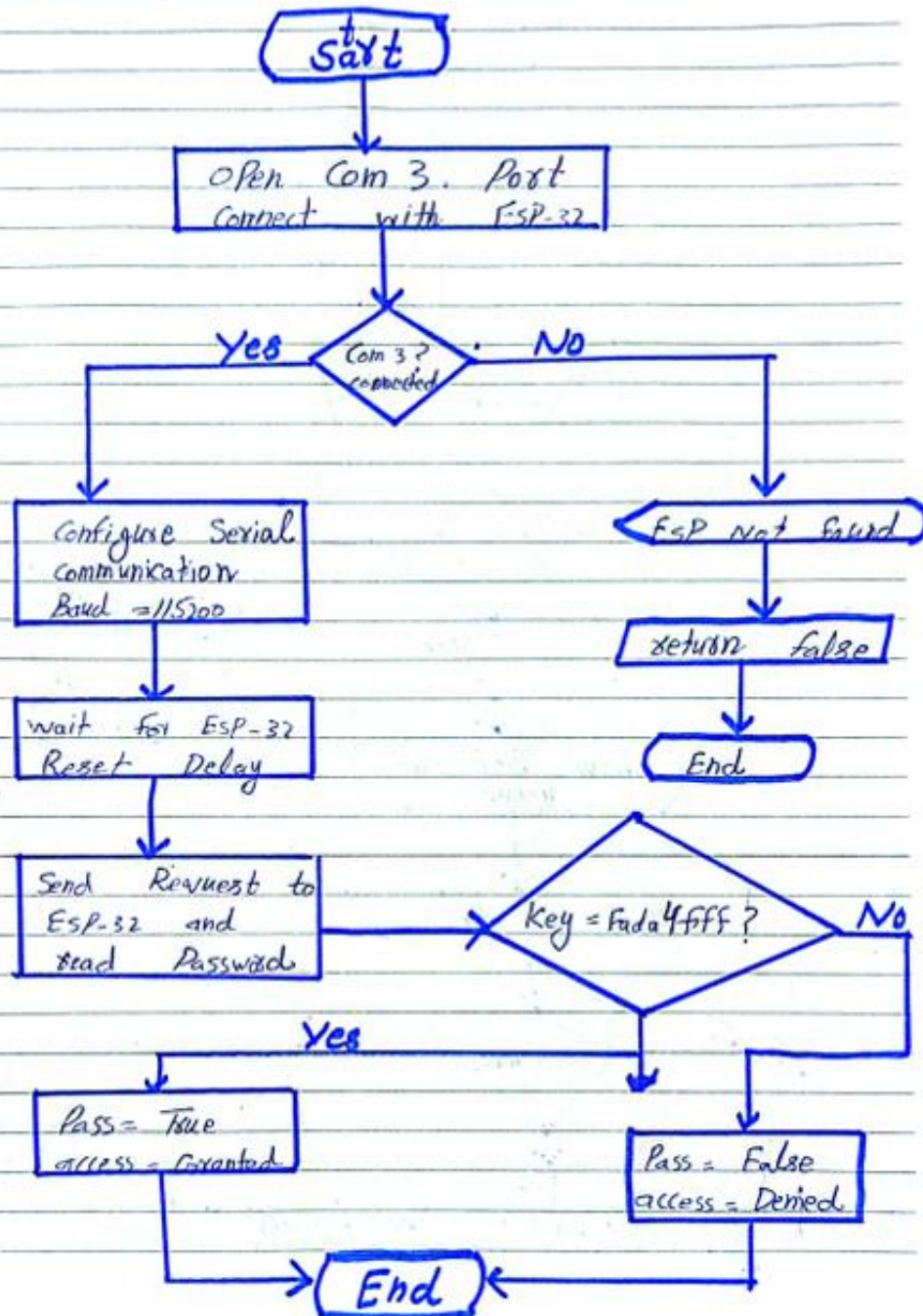
Flow chart

Main function

1

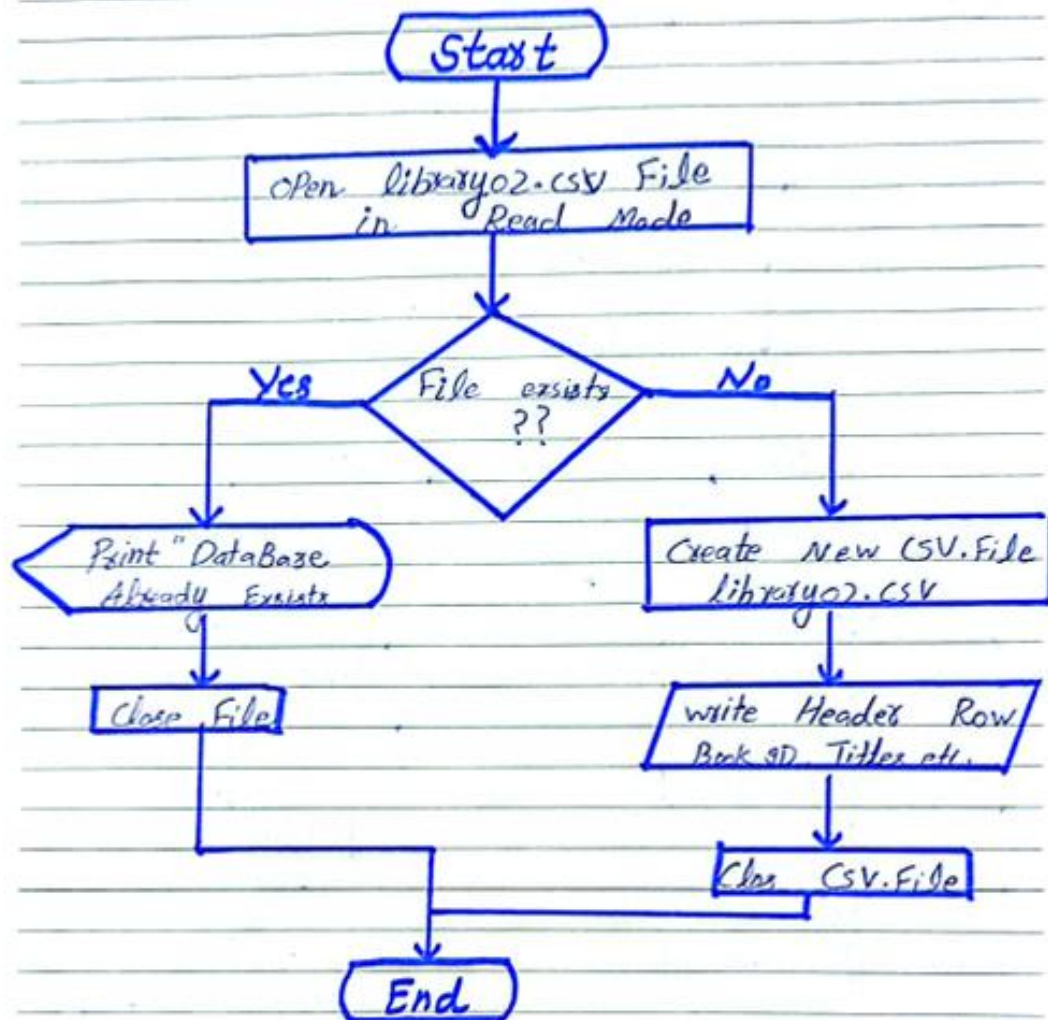


Authenticate ESP-32 () Function

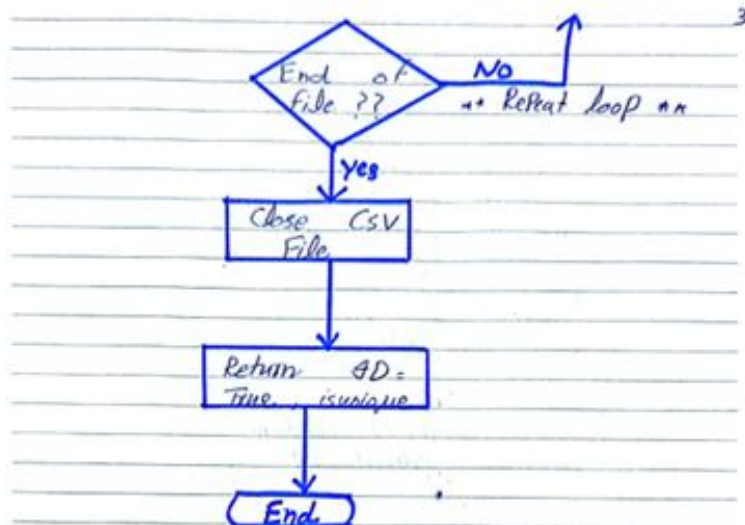
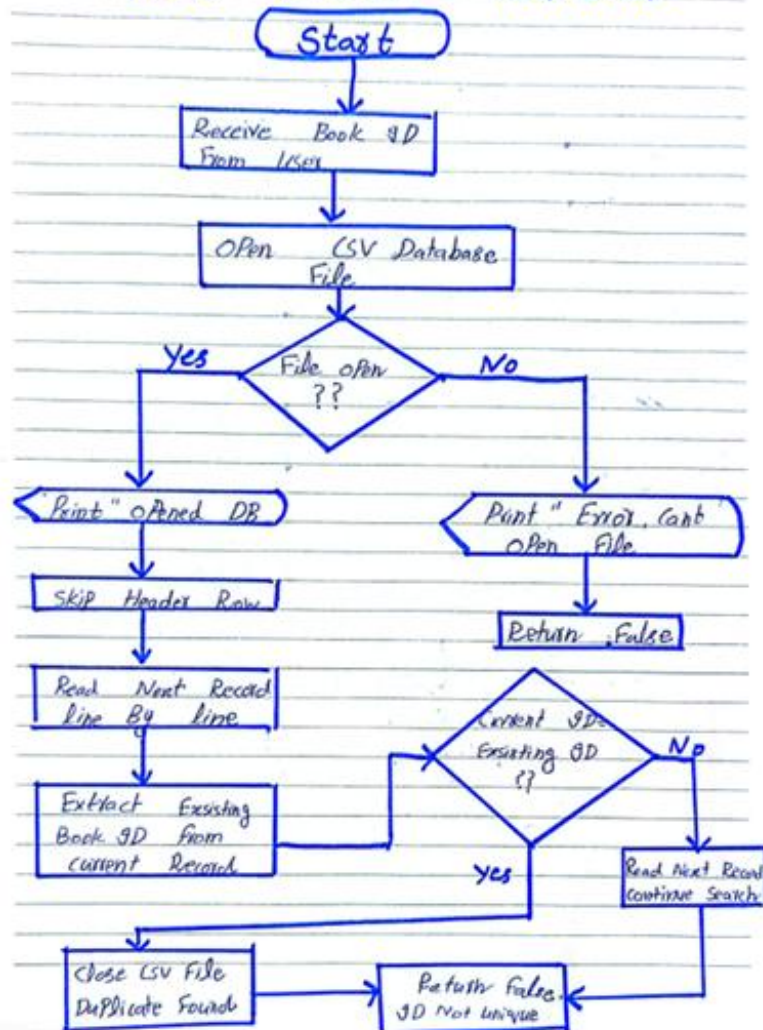


Initialize DataBase () Function

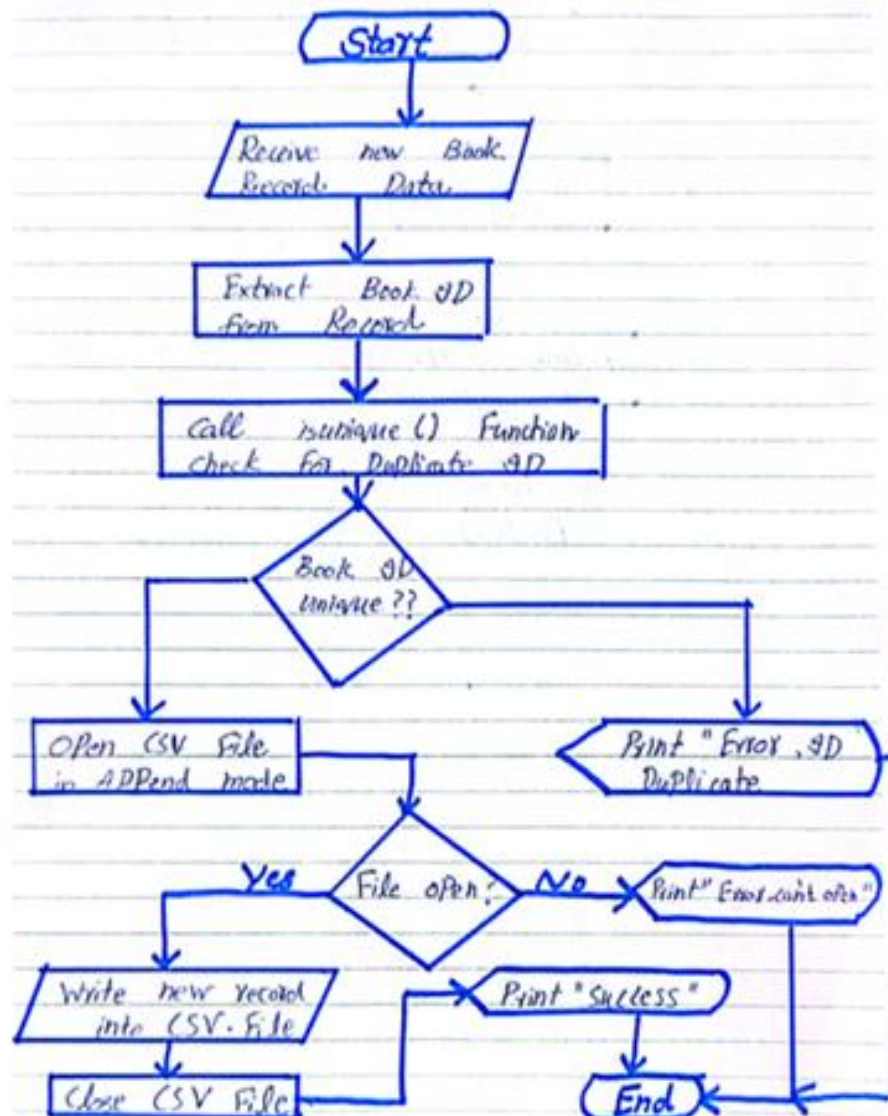
2



is Unique () Function Flow chart

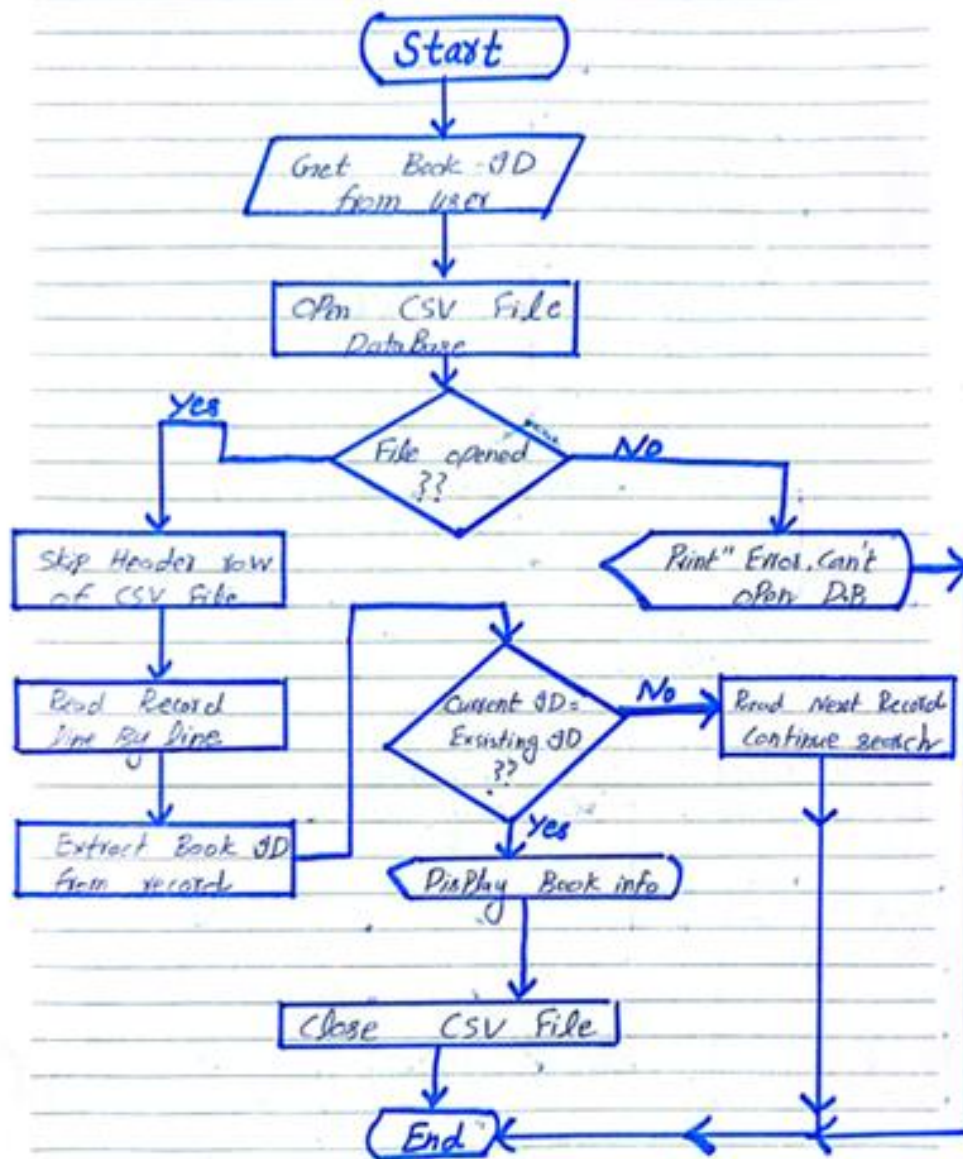


Append Record () Function

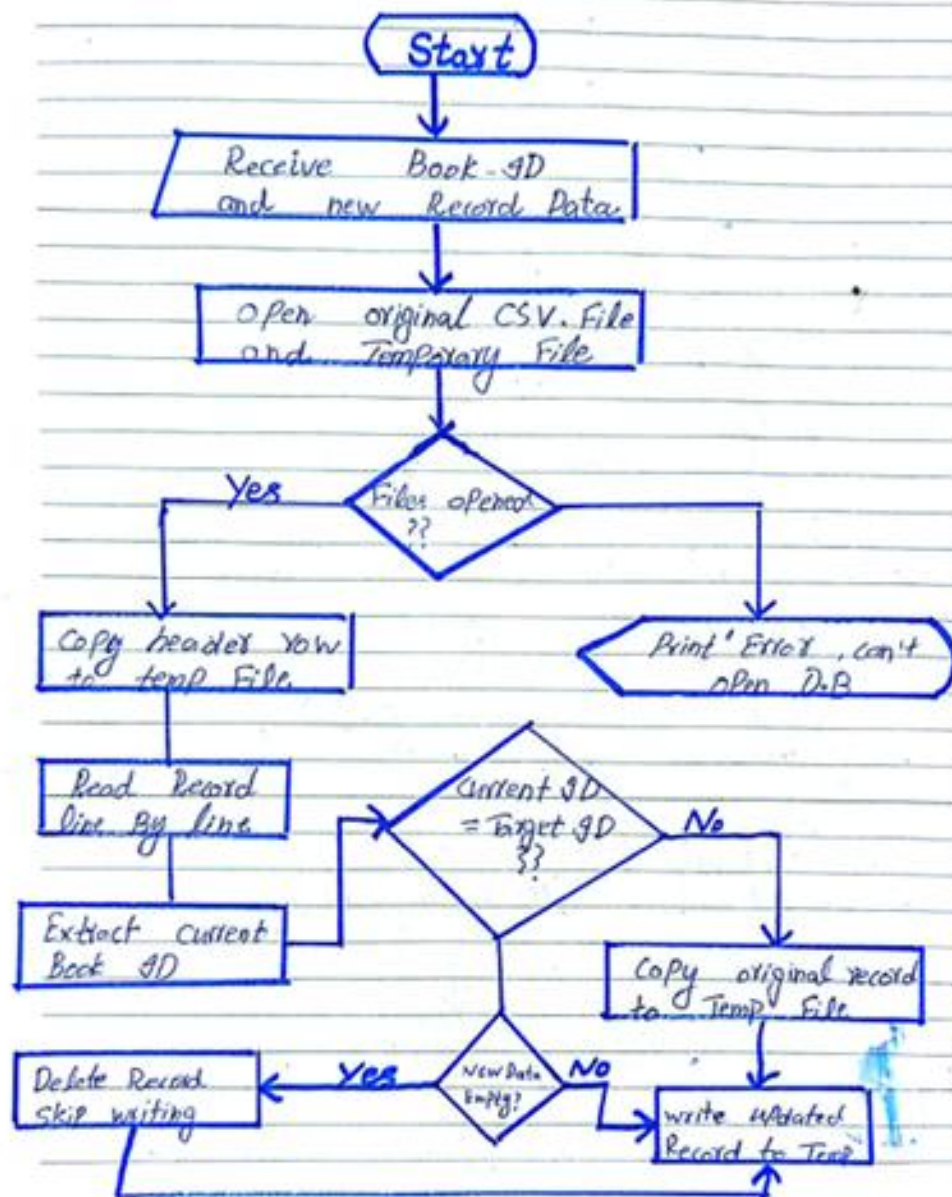


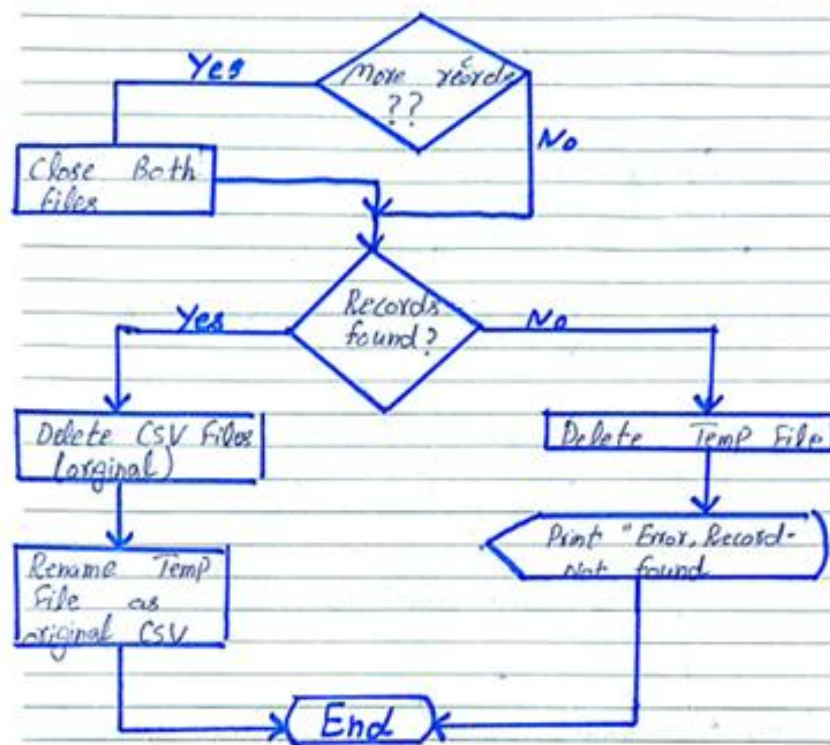
Search By ID() Function

4

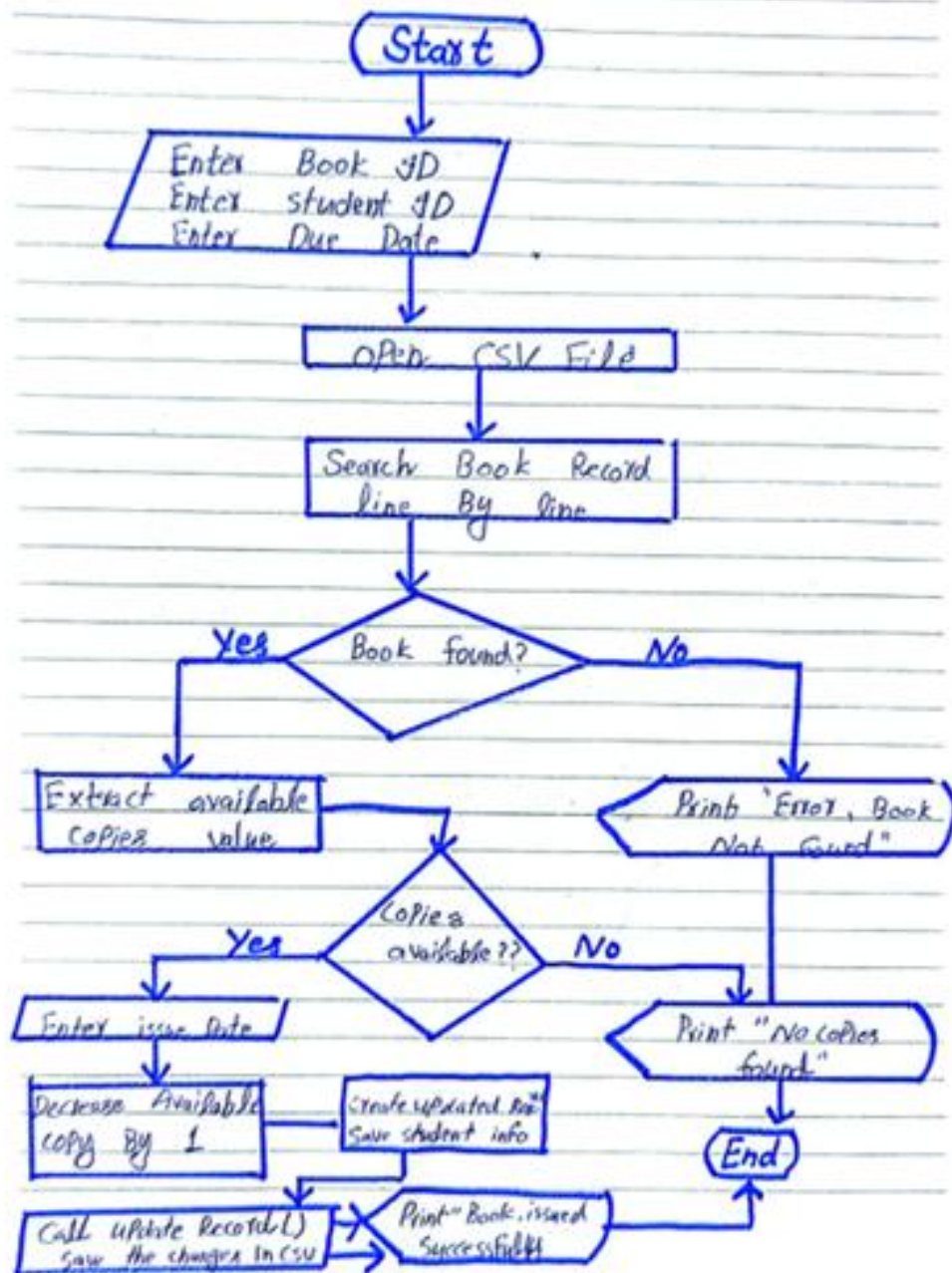


Update Record () Function



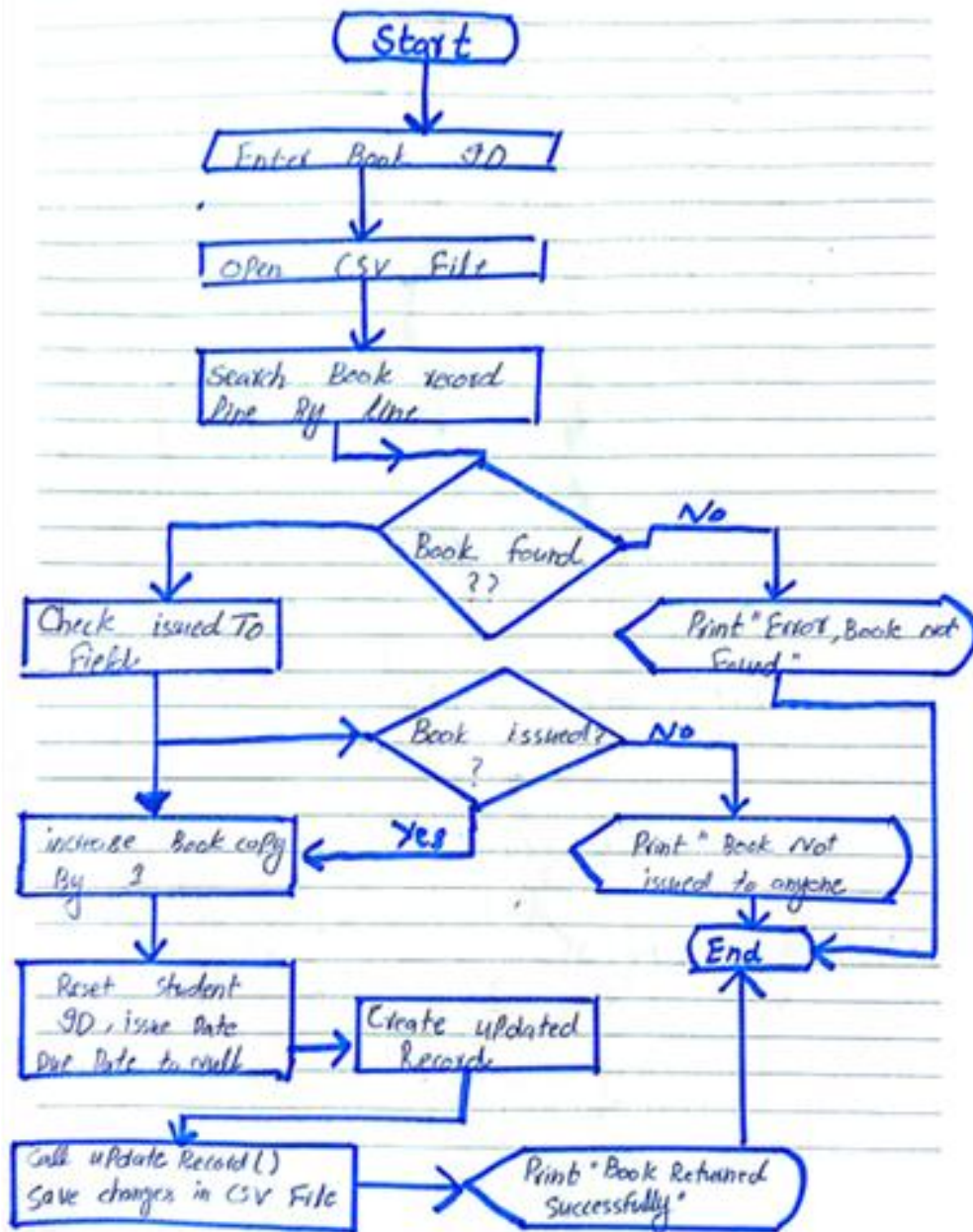


Issue Book () Function

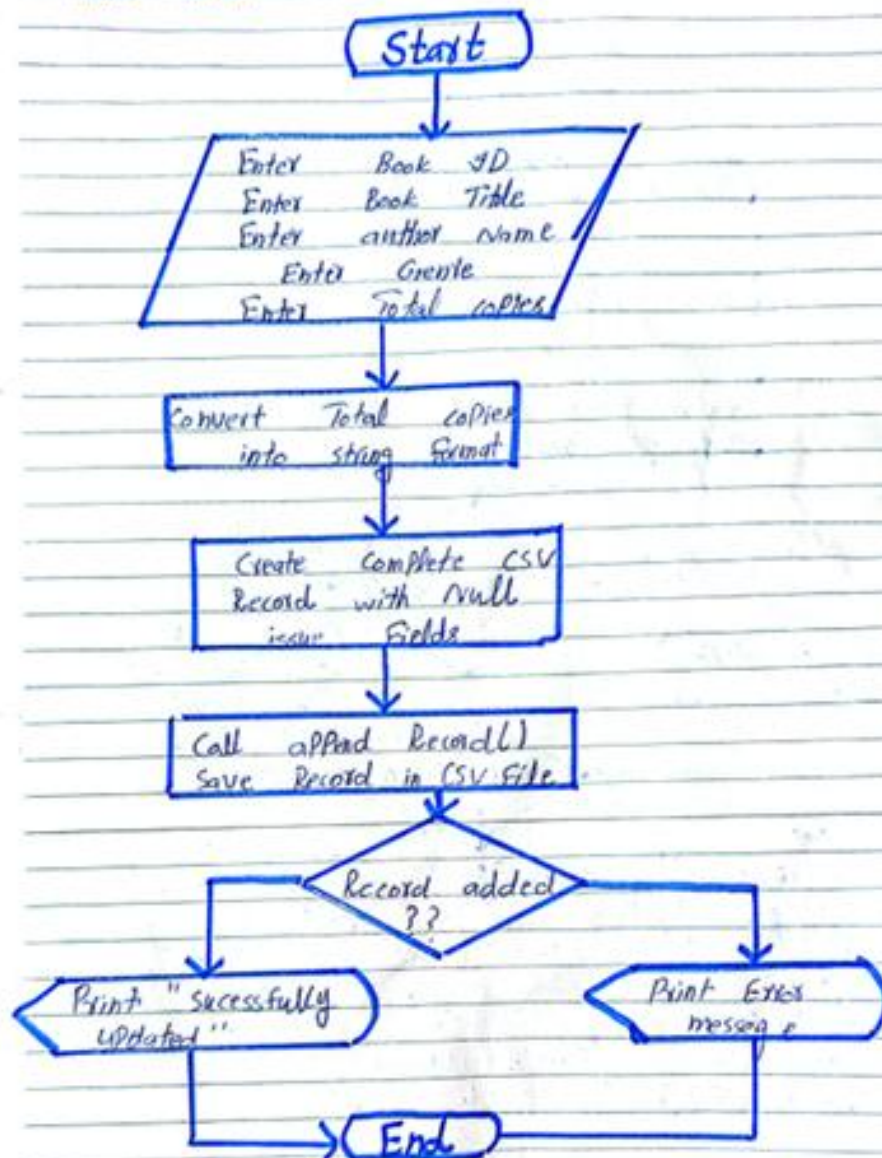


Return Book () Function

6

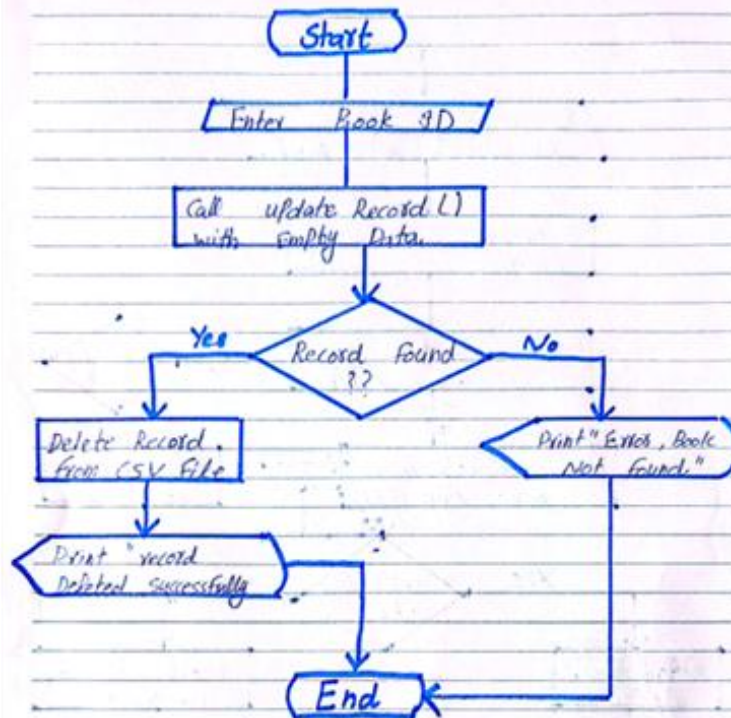


Add Book () Function

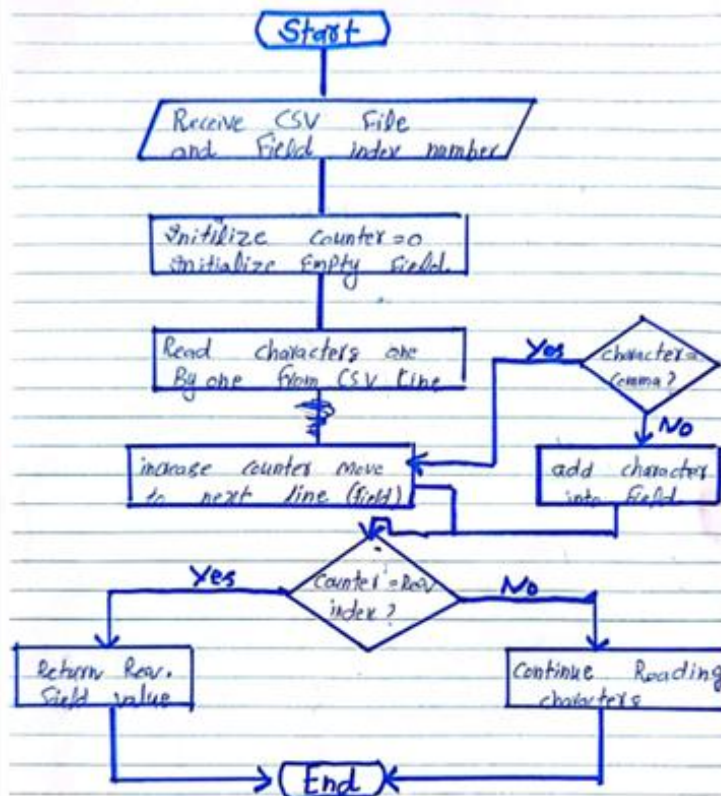


Delete Book () Function

7



Get Field () Function



Testing and Results:

The Library Management System was tested using different test cases to verify the correct working of all features and database operations. Each function was checked individually to ensure proper execution and accurate data management.

Test Case	Expected Output	Result
Add New Book	Record stored successfully	Passed
Duplicate Book ID	Error message displayed	Passed
Search Book	Complete details displayed	Passed
Issue Book	Available copies reduced	Passed
Return Book	Available copies increased	Passed
Delete Book	Record removed successfully	Passed
ESP32 Connected	Access granted	Passed
ESP32 Disconnected	Access denied	Passed

The testing results confirmed that the system performs all operations correctly and maintains records efficiently inside the CSV database.

Advantages and Limitations

Advantages:

- Fast and efficient record management
- Reduces manual paperwork
- Permanent storage of data
- Easy searching and updating of records
- Secure access using ESP32 authentication
- User-friendly menu-driven interface
- Reduces chances of data loss and duplication

Limitations:

- Uses a text-based interface only
- Supports single-user operation
- Uses CSV files instead of advanced SQL databases
- Works only on Windows operating system
- Limited scalability for very large libraries

Future Improvements:

Although the current system works efficiently, several improvements can be made in the future to increase functionality and performance. A graphical user interface (GUI) can be developed to replace the text-based interface and improve user experience.

The CSV database can also be replaced with SQL or MySQL databases for better scalability and faster record management. Additional features such as barcode scanning, RFID-based tracking, fingerprint authentication, and cloud storage integration can also be added.

Multi-user support and online access can make the system more suitable for large educational institutions and modern digital libraries.

Conclusion:

The Library Management System successfully demonstrates the practical implementation of C++ programming, CSV file handling, and ESP32-based authentication in a real-world application. The system efficiently performs important library operations such as adding books, searching records, issuing books, returning books, and deleting records while maintaining data permanently inside a CSV database. The integration of hardware authentication also improves system security and prevents unauthorized access.

This project helped in understanding important programming concepts including file handling, functions, loops, conditional statements, string manipulation, and serial communication. The developed system reduces manual paperwork, improves record management efficiency, and provides a simple and user-friendly solution for small library environments. Overall, the project achieved its objectives successfully and provided valuable practical experience in software and hardware integration.